

Introducción a la programación en Java: sintaxis y aspectos básicos

1º CFGS Desarrollo de Aplicaciones Web

Curso 2023/2024

Profesor: José Miguel Blázquez

Departamento de Informática

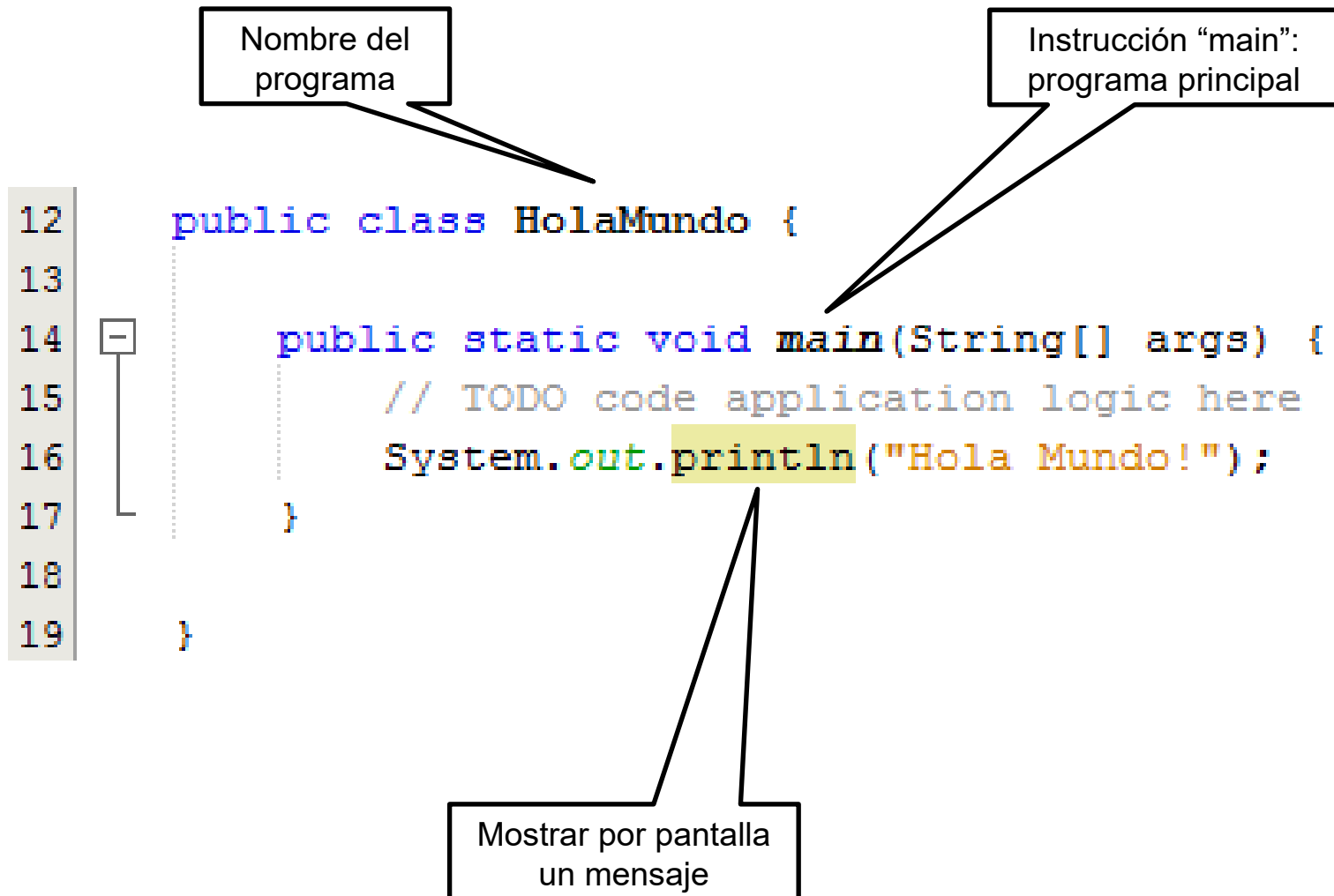


Java

- Lenguaje de prog. de propósito general, concurrente y orientado a objetos.
- **Características:**
 - ✓ Sencillo
 - ✓ Orientado a objetos
 - ✓ Distribuido
 - ✓ Robusto
 - ✓ Seguro
 - ✓ Portable (multiplataforma)
 - ✓ Arquitectura neutral
 - ✓ Rendimiento medio
 - ✓ Multithread



Primer programa



Variables y tipos de datos

Java es un lenguaje “fuertemente tipado”.

Tipos de datos:

- Simples (números, caracteres)
- Compuestos (estructuras, vectores, objetos)

Tipo	Descripción	Memoria ocupada	Rango de valores permitidos
byte	Número entero de 1 byte	1 byte	-128 ... 127
short	Número entero corto	2 bytes	-32768 ... 32767
int	Número entero	4 bytes	-2.147.483.648 ... 2.147.483.647
long	Número entero largo	8 bytes	-9223372036854775808 ... 9223372036854775807
float	Número real en coma flotante de precisión simple	32 bits	$\pm 3,4 \cdot 10^{-38} \dots \pm 3,4 \cdot 10^{38}$
double	Número real en coma flotante de precisión doble	64 bits	$\pm 1,7 \cdot 10^{-308} \dots \pm 1,7 \cdot 10^{308}$
char	Un solo carácter	2 bytes	
boolean	Valor lógico	1 bit	true o false



Variables y tipos de datos

La forma básica de declarar (crear) una variable es la siguiente:

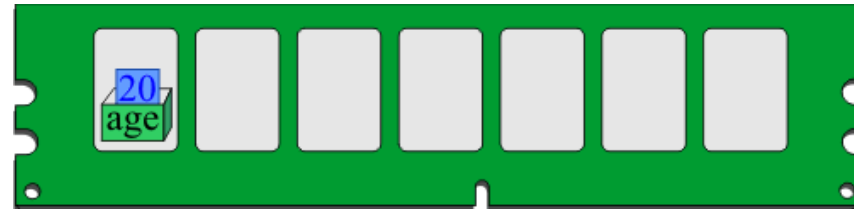
tipo identificador;

Ejemplos:

```
int edad = 20;
```

```
int edad;  
edad = 20;
```

```
float precio1, precio2, precio3;
```



int age = 20;
Type Name Value



00000000	0x0042F2
00000001	0x0042F3
00101001	0x0042F4
01100100	0x0042F5
	0x0042F6
	0x0042F7
	0x0042F8

Operadores

Operador	Formato	Descripción
+	op1 + op2	Suma aritmética de dos operandos.
-	op1 - op2 -op1	Resta aritmética de dos operandos. Cambio de signo.
*	op1 * op2	Multiplicación de dos operandos
/	op1 / op2	División entera de dos operandos
%	op1 % op2	Resto de la división entera (o módulo)
++	++op1 op1++	Incremento unitario
--	--op1 op1--	Decremento unitario

Los operadores ++ y -- realizan un incremento y un decremento unitario respectivamente. Es decir:

- **x++** equivale a **x = x + 1**
- **x--** equivale a **x = x - 1**

Los operadores ++ y -- admiten notación **postfija** y **prefija**:

- **op1++**: Primero se ejecuta la instrucción en la cual está inmerso y después se incrementa op1.
- **op1--**: Primero se ejecuta la instrucción en la cual está inmerso y después se decrementa op1.
- **++op1**: Primero se incrementa op1 y después ejecuta la instrucción en la cual está inmerso.
- **--op1**: Primero se decrementa op1 y después ejecuta la instrucción en la cual está inmerso.



Expresiones y operadores

Ejercicio 1

Determina el valor final almacenado en la variable `result` en cada una de estas expresiones. Puedes escribir un pequeño programa Java más tarde para verificar tus respuestas:

- a) `int result = 4 + 8 * 2 / 4`
- b) `int result = (4 + 8) * 2 / 4`
- c) `int result = (4 + 8) * 3 % 5`

Ejercicio 2

Determina el valor final de la variable `result` después de ejecutar **todas** estas instrucciones:

```
int result = 4;  
result += 3;  
result *= 2;  
result--;  
result %= 4;
```



Operadores

Operador	Formato	Descripción
>	op1 > op2	Devuelve true (cierto) si op1 es mayor que op2
<	op1 < op2	Devuelve true (cierto) si op1 es menor que op2
>=	op1 >= op2	Devuelve true (cierto) si op1 es mayor o igual que op2
<=	op1 <= op2	Devuelve true (cierto) si op1 es menor o igual que op2
==	op1 == op2	Devuelve true (cierto) si op1 es igual a op2
!=	op1 != op2	Devuelve true (cierto) si op1 es distinto de op2

```
15 public static void main(String[] args){
16
17     double op1,op2;
18     char op3,op4;
19
20     op1=1.34;
21     op2=1.35;
22     op3='a';
23     op4='b';
24
25     System.out.println("op1=" + op1 + " op2=" + op2);
26     System.out.println("op1>op2 = " + (op1 > op2));
27     System.out.println("op1<op2 = " + (op1 < op2));
28     System.out.println("op1==op2 = " + (op1 == op2));
29     System.out.println("op1!=op2 = " + (op1 != op2));
30     System.out.println("'a'>'b' = " + (op3 > op4));
31
32 }
```


Operadores

Operador	Formato	Descripción
&&	op1 && op2	Y lógico. Devuelve true (cierto) si son ciertos op1 y op2
	op1 op2	O lógico. Devuelve true (cierto) si son ciertos op1 o op2
!	! op1	Negación lógica. Devuelve true (cierto) si es false op1.

```
15 public static void main(String[] args){
16
17     boolean a, b, c, d;
18
19     a=true;
20     b=true;
21     c=false;
22     d=false;
23
24     System.out.println("true Y true = " + (a && b) );
25     System.out.println("true Y false = " + (a && c) );
26     System.out.println("false Y false = " + (c && d) );
27     System.out.println("true O true = " + (a || b) );
28     System.out.println("true O false = " + (a || c) );
29     System.out.println("false O false = " + (c || d) );
30     System.out.println("NO true = " + !a);
31     System.out.println("NO false = " + !c);
32     System.out.println("(3 > 4) Y true = " + ((3 >4) && a) );
33
34 }
35 }
```

Operadores

Precedencia de operadores

Operador	Formato	Equivalencia
+=	op1 += op2	op1 = op1 + op2
-=	op1 -= op2	op1 = op1 - op2
*=	op1 *= op2	op1 = op1 * op2
/=	op1 /= op2	op1 = op1 / op2
%=	op1 %= op2	op1 = op1 % op2
&=	op1 &= op2	op1 = op1 & op2
=	op1 = op2	op1 = op1 op2
^=	op1 ^= op2	op1 = op1 ^ op2
>>=	op1 >>= op2	op1 = op1 >> op2
<<=	op1 <<= op2	op1 = op1 << op2
>>>=	op1 >>>= op2	op1 = op1 >>> op2

- 1) Operadores postfijos: `[]` . (paréntesis)
- 2) Operadores unarios: `++expr`, `--expr`, `-expr`, `~ !`
- 3) Creación o conversión de tipo: `new (tipo)expr`
- 4) **Multiplicación y división: `*`, `/`, `%`**
- 5) **Suma y resta: `+`, `-`**
- 6) Desplazamiento de bits: `<<`, `>>`, `>>>`
- 7) **Relacionales: `<`, `>`, `<=`, `>=`**
- 8) **Igualdad y desigualdad: `==`, `!=`**
- 9) AND a nivel de bits: `&`
- 10) **AND lógico: `&&`**
- 11) XOR a nivel de bits: `^`
- 12) OR a nivel de bits: `|`
- 13) **OR lógico: `||`**
- 14) Operador condicional: `?` :
- 15) **Asignación: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `^=`, `&=`, `|=`, `>>=`, `<<=`**

Operaciones básicas de salida

```
public class Main
{
    public static void main(String[] args)
    {
        int dias = 7; int hora = 12;

        System.out.println("Bienvenido/a.");
        System.out.println("La semana tiene " + dias + " dias.");
        System.out.println("Son las " + hora + " en punto");
        System.out.println("Luego serán las " + hora + 2 + " pm");
        System.out.println("Luego serán las " + (hora + 2) + " pm");
    }
}
```

Salida:

```
Bienvenido/a.
La semana tiene 7 dias.
Son las 12 en punto
Luego serán las 122 pm
Luego serán las 14 pm
```

Las palabras entre comillas se muestran directamente. El resto se procesa.



Operaciones de salida (II)

```
public class Main
{
    public static void main(String[] args)
    {
        System.out.println("Esta es la frase 1.");
        System.out.print("Esta es la frase 2.");
        System.out.print("Esta es la frase 3.");
    }
}
```

¿Cuál será la salida por pantalla?....

Salida:

```
Esta es la frase 1.
Esta es la frase 2.Esta es la frase 3.
```

print("...") : imprime texto por pantalla

println("...") : imprime texto por pantalla e introduce un salto de línea.



Operaciones básicas de entrada

```
import java.util.Scanner;
```

```
public class Main  
{
```

```
    public static void main(String[] args)  
    {
```

```
        int dias;
```

```
        Scanner teclado = new Scanner(System.in);
```

```
        System.out.println("¿Cuántos días tiene julio?");
```

```
        dias = teclado.nextInt();
```

```
        System.out.println("Dices que julio tiene: " + dias + " días.");
```

```
    }
```

```
}
```

Para leer datos de teclado

```
graph TD; A[Para leer datos de teclado] --> B[import java.util.Scanner;]; A --> C[Scanner teclado = new Scanner(System.in);]; A --> D[dias = teclado.nextInt();];
```

Escribe y prueba este programa. Ejecútalo varias veces y cambia el valor.

Operaciones básicas de entrada (II)

Otros métodos importantes del objeto Scanner:

- .nextFloat():** obtiene un numero real float.
- .nextLine():** recogerá el texto que el usuario escriba por teclado (hasta presionar la tecla Intro).
- .next():** obtiene el siguiente token (texto hasta un espacio).

Veamos ejemplos para leer una frase o un decimal (declarando antes las variables...)

```
int edad; String nombre;  
Scanner leer = new Scanner(System.in);  
  
nombre = leer.nextLine();  
edad = leer.nextInt();
```

```
int edad; String nombre;  
Scanner leer = new Scanner(System.in);  
  
edad = leer.nextInt();  
nombre = leer.nextLine();
```



Altera el orden de lectura, primero el valor del peso y luego el nombre, ¿qué sucede?



Lectura de datos diferentes

Cuidado al combinar tipos de datos. Supongamos que tiene que leer esta información de la entrada:

23 43

Hello world

Posible secuencia:



```
int number1 = sc.nextInt();  
int number2 = sc.nextInt();  
String text = sc.nextLine();
```

Solución:

```
int number1 = sc.nextInt();  
int number2 = sc.nextInt();  
sc.nextLine();  
String text = sc.nextLine();
```

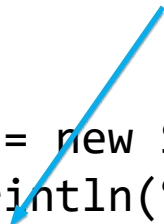


Limpiar el buffer del stream

2 formas básicas:

- Cutre, pero rápida
Intercalar un “.nextLine()” tras la lectura de un número (o un .next()), leerá el “\r” y el buffer quedará limpio para seguir.
- Profesional (Desarrollador)
Leer todo con nextLine y convertirlo al tipo de datos destino:

```
int edad;  
Scanner leer = new Scanner(System.in);  
System.out.println("Introduce tu edad: ");  
edad = Integer.parseInt(leer.nextLine());
```

A blue arrow points from the 'leer' variable in the Scanner declaration to the 'leer' parameter in the parseInt method call. A blue box highlights the 'Integer.parseInt' part, and a red box highlights the 'leer.nextLine()' part.

El “parsing” permite convertir texto en otros formatos mediante el procesamiento. Se puede hacer con Float, Double, etcétera...



Estructura básica programa

```
/* ****  
Programa para calcular el área de un círculo a partir del radio  
**** */
```

```
import java.util.Scanner;
```

```
public class Main  
{
```

```
    public static void main(String[] args)  
    {
```

```
        float radio; float area;  
        Scanner leer = new Scanner(System.in);
```

Declarar variables

```
        System.out.println("Introduce el valor del radio: ");  
        radio = leer.nextFloat();
```

Pedir dato y leerlo

```
        area = 3.1416f * radio * radio;
```

Calcular el área

```
        System.out.println("El área es: " + area + " cm2.");
```

Mostrar el resultado

```
    }
```

```
}
```

Ejercicios propuestos

- 1) Programa para calcular el área de un triángulo (pide al usuario los datos que necesites).
- 2) Programa para calcular el volumen de una esfera (pide al usuario los datos que necesites).
- 3) Programa para convertir de grados Fahrenheit a grados centígrados, y viceversa.
- 4) Programa que pida las notas de los 3 trimestres y calcule la nota media final.
- 5) Programa para calcular el PH de una piscina según:
$$\text{pH} = -\log_{10}[\text{H}_3\text{O}^+]$$



Constantes

Creación de variable constante tipo float llamada pi con valor 3.14:

```
final float PI = 3.1416;
```

Si posteriormente intentamos modificar el valor se producirá un error y Java nos avisará de que no es posible.

PI = 7; // no permitido, produce error

Este tipo de valores sólo se van a utilizar, generalmente, en elementos que son de naturaleza invariable: constantes físicas o matemáticas...



Librerías: la clase Math

Posee muchos métodos para realizar cálculos matemáticos complejos como potencias, raíces cuadradas, valores absolutos, funciones trigonométricas, etcétera...

```
double x = Math.pow(3,2); // Potencia 3 ^ 2  
double y = Math.sqrt(9); // Raíz cuadrada de 9
```

También posee constantes como:

double PI -> El número π (3,14159265...)

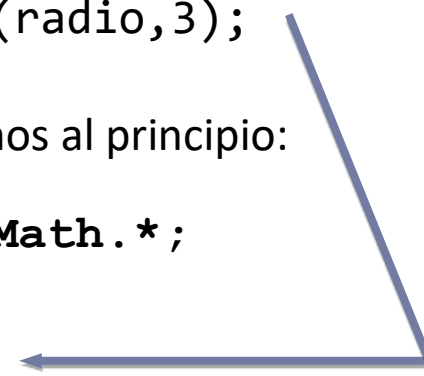
double E -> El número e (2, 7182818245...)

```
area = 4/3.0*Math.PI*Math.pow(radio,3);
```

Podemos omitir el nombre de la clase si la importamos al principio:

```
import static java.lang.Math.*;
```

```
area = 4/3.0*PI*pow(radio,3);
```



Secuencias de escape

Son combinaciones del símbolo contrabarra \ seguido de una letra, y sirven para representar caracteres que no tienen una equivalencia en forma de símbolo.

Utilizados mayormente en instrucciones `System.out.println`

Secuencia	Significado
\n	Nueva Linea.
\t	Tabulador
\"	Dobles comillas.
\'	Comilla simple.
\\	Barra invertida.

```
System.out.println("Hola");  
System.out.print("Hola\n");
```

¡Son equivalentes!



Ejercicios propuestos (II)

5) Programa que pida 2 números al usuario y que muestre, en líneas separadas, la suma, resta, multiplicación y división.

6) Programa que lea un precio de un producto y devuelva el valor final con IVA (21%) incluido, en euros.

7) Aplicación que resuelva una ecuación de segundo grado. Debes pedir los coeficientes **a**, **b** y **c** por teclado. Para la raíz cuadrada usa el método **sqrt** de **Math**. **Ojo**: hay 2 soluciones...



Conversión de tipos

Las conversiones entre valores numéricos son bastante sencillas. Solo falta hacer un *typecast*: “especificar entre paréntesis el tipo de dato al que queremos convertir la expresión”. En este ejemplo, estamos convirtiendo el valor real en un número entero (por lo que obtenemos 3 como resultado final):

```
float pi = 3.1416f;  
int piInteger = (int)pi;
```

También se puede hacer el paso contrario. En este caso, estamos convirtiendo un valor entero en uno doble (el valor final será 5,0):

```
int number = 5;  
double realNumber = (double)number;
```

Sin embargo, este paso NO es necesario si el tipo de origen es más pequeño que el tipo de destino. Por ejemplo, a byte no necesita convertirse en int:

```
byte value = 3;  
int number = value;
```



Conversión de tipos

Typecast puede ser útil, por ejemplo, para convertir divisiones **enteras** en divisiones **reales**. Este ejemplo divide 2 enteros, pero al convertir uno de ellos en *float*, entonces el resultado final será un número *real*, con los dígitos de fracción correspondientes, y se puede almacenar en una variable float:

```
float result = (float) 3 / 2;
```

```
area = (base*altura)/2.0f
```

```
area = (float)((base*altura)/2.0)
```

Conversión mediante el literal “f”. Fuerza un decimal tipo “double” (por defecto en Java) a un decimal tipo “float”. Además, todo se convierte a *float* por ser “mayor” que *int*.

Conversión forzada de toda la expresión, del tipo “double” al tipo “float”. Atención: hay que poner toda la expresión a convertir **entre paréntesis**.

Limitar decimales en pantalla

```
import java.math.RoundingMode;  
import java.text.DecimalFormat;
```

```
DecimalFormat decimales = new DecimalFormat("0.###");  
decimales.setRoundingMode(RoundingMode.DOWN);
```

```
System.out.println("Resultado: " + decimales.format(N));
```

Si no empleamos el método “setRoundingMode” lo que obtendremos será el redondeo.

Experimenta otros patrones en DecimalFormat...

Investiga: ¿cómo puedes redondear o truncar usando la clase Math?

Pista: .ceil() .floor() .round()



Diagrama vs código Java

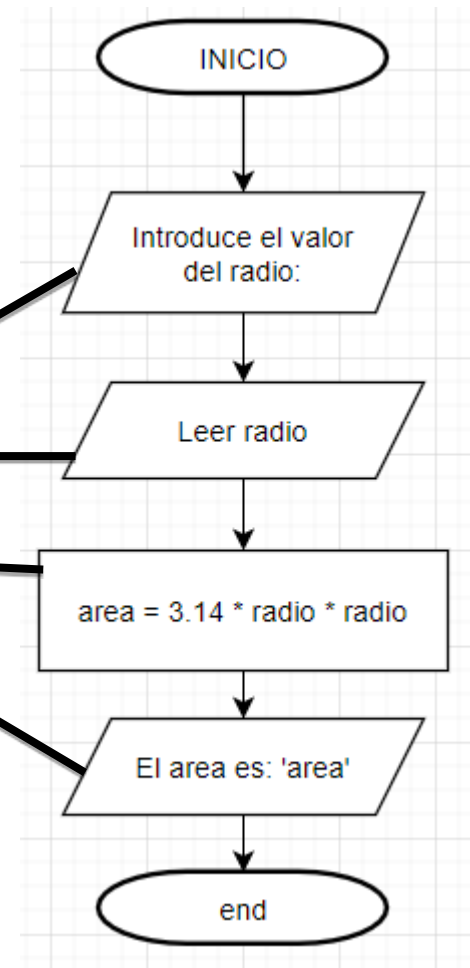
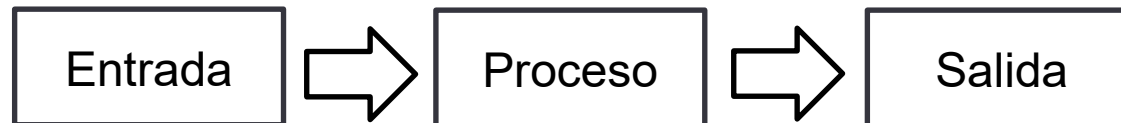
```
import java.util.Scanner;
```

```
public class Main
{
    public static void main(String[] args)
    {
        float radio; float area;
        Scanner leer = new Scanner(System.in);

        System.out.println("Introduce el valor del radio: ");
        radio = leer.nextFloat();

        area = 3.1416f * radio * radio;

        System.out.println("El área es: " + area + " cm2.");
    }
}
```



Ejercicio extra

Crea un programa llamado *NumbersStrings*.

Este programa debe solicitar al usuario que introduzca 4 números, que serán almacenados en 4 variables *String*.

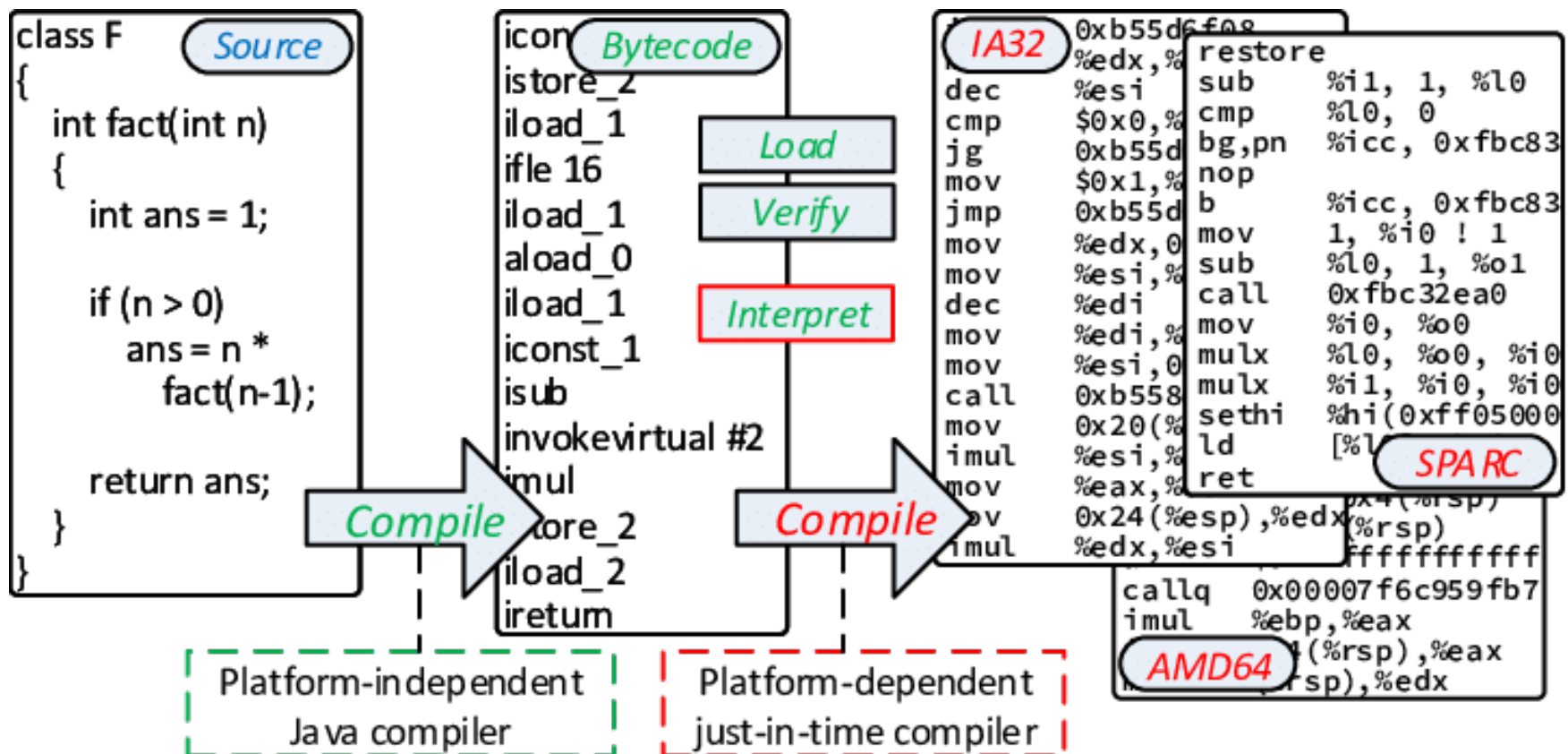
Luego, el programa unirá el primer par de números en un solo valor entero, y el segundo par de números en otro valor entero. Tras esto, sumará estos valores.

Por ejemplo, si el usuario escribe los números 23, 11, 45 y 112, el programa creará un 1er valor entero 2311 y un 2º valor entero 45112.

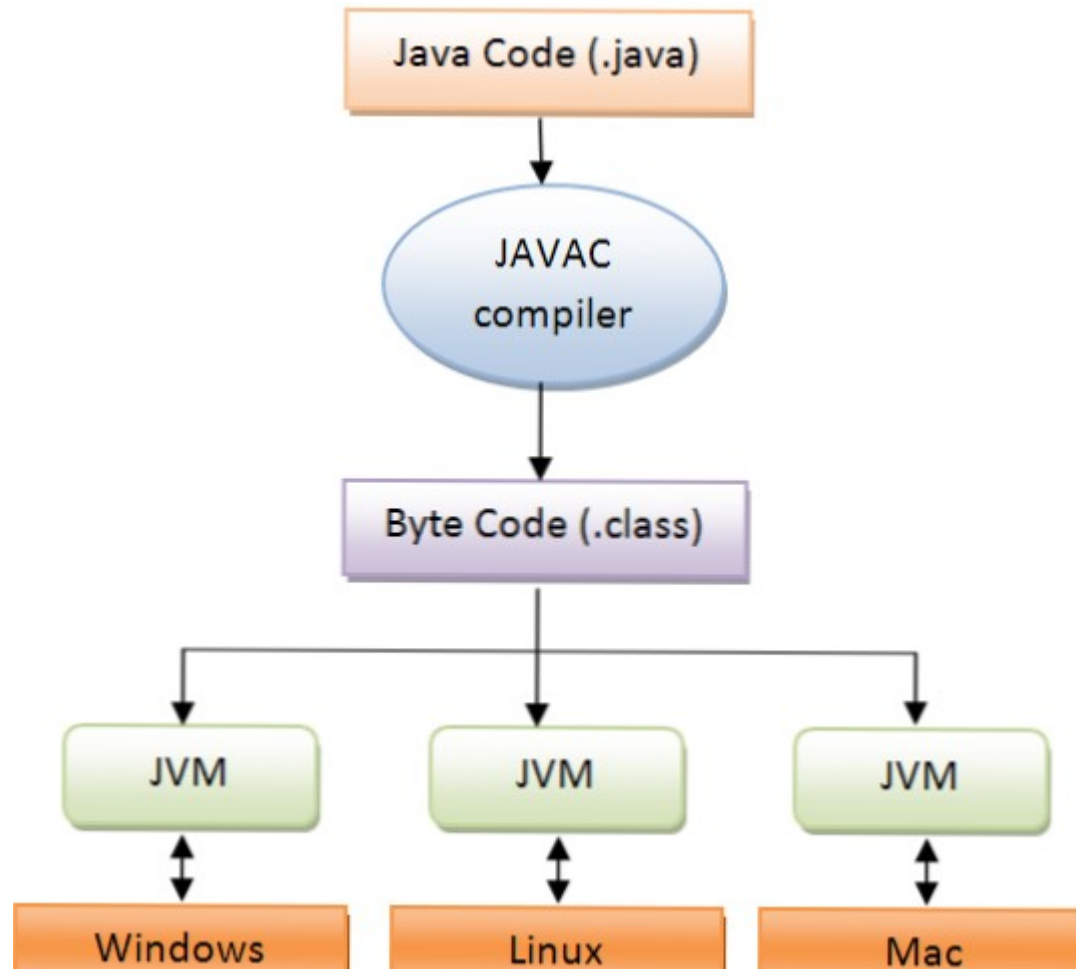
Al final, sumará estos dos valores y obtendrá un resultado de 47423.



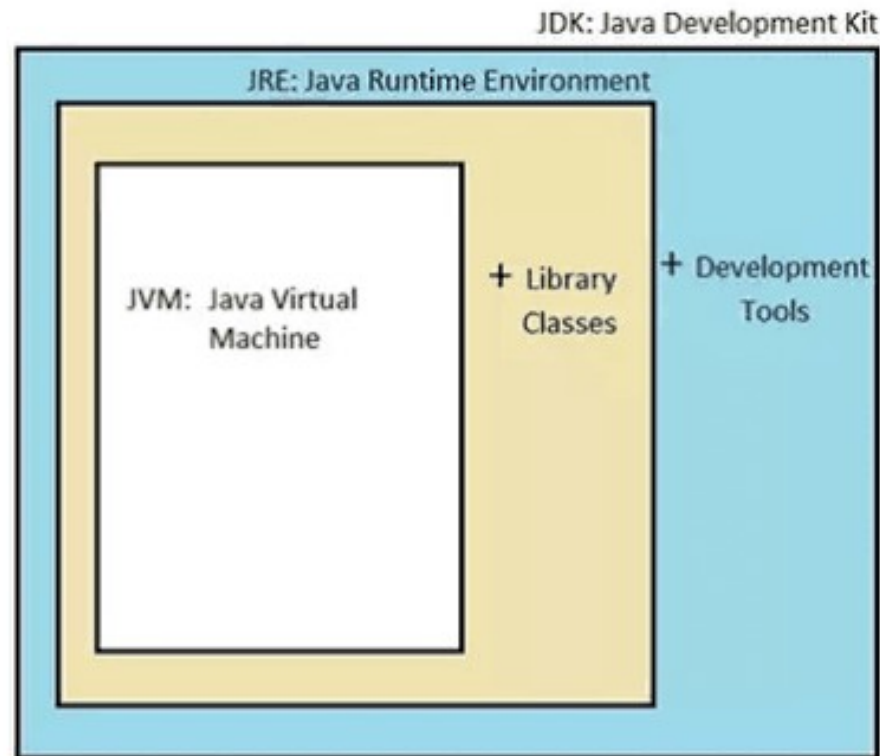
Aspectos técnicos JVM



Aspectos técnicos JVM



Aspectos técnicos Java



JDK = JRE + Development Tools

JRE = JVM + Library Classes



Ejercicio final unidad

9) Realiza un programa calcule el valor de estas expresiones según los datos que se te proporcionan.

$$a = 2 \quad b = 5$$

$$x = \frac{a + b}{b + a + 1} \cdot \sqrt{a^3}$$

$$y = \frac{a^2 + 2ab + b^2}{\frac{1}{a} + 2}$$

$$z = \ln e^2 - (a + b^3)^2 + \frac{2 + b}{\sqrt[3]{(a + 2b)^2}}$$

Debe salir por pantalla **exactamente** lo que se ve debajo:

Programa del ejercicio 8.
Este se complica un poco...

El resultado de x es: 2,4748
El resultado de y es: 19,6
El resultado de z es: -16125,6645

Los valores de x, y, z se obtienen con las expresiones de arriba y los valores de a y b indicados.

